

- 1) Compare the performance of a linear queue and a Circular queue when 100 enqueue and dequeue operations are performed alternately.

Step-1: Understand the Situation

* We have a queue.

* perform 100 operations alternately.

> Enqueue (Insert)

> Dequeue (Delete)

> Enqueue

> Dequeue

Step-2: Linear Queue

Suppose the queue size is 5.

Initially

Queue: [][][][][]

Front = -1

Rear = -1

Dequeue

Delete 10

Queue: [][30][][][]

Front = 1

Rear = 1

Enqueue 10

Queue: [10][][][][]

Front = 0

Rear = 0

Enqueue 20

Queue: [10][20][][][]

Front = 0

Rear = 1

∴ The first position becomes empty.

Continue Operations

Queue: [][][30][40][50]

Front = 2

Rear = 4

Even though the first two positions are empty.

[][][30][40][50]

Result:

Queue - full

Step-3: performance of Linear Queue

- > Empty Spaces at the beginning cannot be reused.
- > After many enqueue and dequeue operations, the rear reaches the last position.
- > Queue becomes full even when empty spaces exist.
- > Memory is wasted.

Step-4: Circular Queue

Again assume queue size = 5.

Initially

[10][20][30][40][50]

front = 0

rear = 4

Delete two elements.

[][][30][40][50]

front = 2

rear = 4

Now insert 60.

Instead of saying "Queue full".

[60][][30][40][50]

rear = 0

front = 2

Insert 70

[60][70][30][40][50]

rear = 1

front = 2

∴ The empty spaces are reused.

∴ circular queue performs better because it reuses empty spaces and avoids unnecessary queue full conditions.

queue implemented using an array reports "Queue full" even though some positions are empty.

a) Analyze why this occurs.

Step: 1 - consider a Queue of size 5

Initially

[10][20][30][40][50]

front = 0

Rear = 4

Step: 2 - Delete two elements

[][][30][40][50]

front = 2

rear = 4

Now positions 0 and 1 are empty.

Step-3: Try to Insert Another element

Rear is already at index 4.

The insertion condition is

$$\text{Rear} == \text{Size} - 1$$

> which is true.

Therefore,

Queue full is displayed,

Step-4: Why does this happen?

Because in a Linear queue:

> Rear moves only forward.

> Rear never comes back.

> Hence memory is wasted.

Conclusion:

A linear Queue may report Queue full even when some positions are empty because the rear has reached end.

b) Explain how a circular Queue solves this problem.

Step-1: Initial Queue

[10][20][30][40][50]

Front = 0

Rear = 4

Step-2: Delete two elements

[][][30][40][50]

front = 2

rear = 4

Step-3: Insert 60

Instead of Stopping,

Rear wraps around.

$Rear = (rear + 1) \% \text{Size}$

$rear = (4 + 1) \% 5 = 0$

Queue becomes

[60][][30][40][50]

Rear = 0

front = 2

Step-4: Insert 70

$Rear = (0 + 1) \% 5 = 1$

$\Rightarrow [60][70][30][40][50]$

rear = 1

front = 2

Step-5: How Circular Queue solves the problem

* Empty Spaces are reused

* No memory is wasted.

$\therefore$ A Circular queue is more efficient than a linear queue because it allows the rear pointer to wrap around and reuse empty spaces.